WIKIPEDIA 25
25 years of the free encyclopedia

Serverless computing

Serverless computing is "a cloud service category where the customer can use different cloud capability types without the customer having to provision, deploy and manage either hardware or software resources, other than providing customer application code or providing customer data. Serverless computing represents a form of virtualized computing", according to ISO/IEC 22123-2.[1] Serverless computing is a broad ecosystem that includes the cloud provider, function as a service (FaaS), managed services, tools, frameworks, engineers, stakeholders, and other interconnected elements.[2]

Overview

Serverless is a misnomer in the sense that servers are still used by cloud service providers to execute code for developers. The definition of serverless computing has evolved over time, leading to varied interpretations. According to Ben Kehoe, serverless represents a spectrum rather than a rigid definition. Emphasis should shift from strict definitions and specific technologies to adopting a serverless mindset, focusing on leveraging serverless solutions to address business challenges.[3]

Serverless computing does not eliminate complexity but shifts much of it from the operations team to the development team. However, this shift is not absolute, as operations teams continue to manage aspects such as identity and access management (IAM), networking, security policies, and cost optimization. Additionally, while breaking down applications into finer-grained components can increase management complexity, the relationship between granularity and management difficulty is not strictly linear. There is often an optimal level of modularization where the benefits outweigh the added management overhead.[4][2]

According to Yan Cui, serverless techniques should be adopted only when they help to deliver customer value faster. And while adopting, organizations should take small steps and de-risk along the way.[5]

Challenges

Serverless applications are prone to fallacies of distributed computing. In addition, they are prone to the following fallacies:[6][7]

- Versioning is simple
- Compensating transactions always work
- Observability is optional

Monitoring and debugging

Monitoring and debugging serverless applications can present unique challenges due to their distributed, event-driven nature and proprietary environments. Traditional tools may fall short, making it difficult to track execution flows across services. However, modern solutions such as distributed tracing tools (e.g., AWS X-Ray, Datadog), centralized logging, and cloud-agnostic observability platforms are mitigating these challenges. Emerging technologies like OpenTelemetry, AI-powered anomaly detection, and serverless-specific frameworks are further improving visibility and root cause analysis. While challenges persist, advancements in monitoring and debugging tools are steadily addressing these limitations.[8][9]

Security

According to OWASP, serverless applications are vulnerable to variations of traditional attacks, insecure code, and some serverless-specific attacks (like *denial of wallet*[10]). So, the risks have changed and attack prevention requires a shift in mindset.[11][12]

Vendor lock-in

Serverless computing is provided as a third-party service. Applications and software that run in the serverless environment are by default locked to a specific cloud vendor. This issue is exacerbated in serverless computing, as with its increased level of abstraction, public vendors only allow customers to upload code to a FaaS platform without the authority to configure underlying environments. More importantly, when considering a more complex workflow that includes backend-as-a-service (BaaS), a BaaS offering can typically only natively trigger a FaaS offering from the same provider. This makes the workload migration in serverless computing virtually impossible. Therefore, considering how to design and deploy serverless workflows from a multi-cloud perspective could mitigate this.[13][14][15]

High-performance computing

Serverless computing may not be ideal for certain high-performance computing (HPC) workloads due to resource limits often imposed by cloud providers, including maximum memory, CPU, and runtime restrictions. For workloads requiring sustained or predictable resource usage, bulk-provisioned servers can sometimes be more cost-effective than the pay-per-use model typical of serverless platforms. However, serverless computing is increasingly capable of supporting specific HPC workloads, particularly those that are highly parallelizable and event-driven, by leveraging its scalability and elasticity. The suitability of serverless computing for HPC continues to evolve with advancements in cloud technologies.[16][17][18]

Anti-patterns

The *grain of sand* anti-pattern refers to the creation of excessively small components (e.g., functions) within a system, often resulting in increased complexity, operational overhead, and performance inefficiencies.[19] *Lambda pinball* is a related anti-pattern that can occur in serverless architectures when functions (e.g., AWS Lambda, Azure functions) excessively invoke each other in fragmented chains, leading to latency, debugging and testing challenges, and reduced observability.[20] These anti-patterns are associated with the formation of a distributed monolith.

These anti-patterns are often addressed through the application of clear domain boundaries, which distinguish between public and published interfaces.[20][21] Public interfaces are technically accessible interfaces, such as methods, classes, API endpoints, or triggers, but they do not come with formal stability guarantees. In contrast, published interfaces involve an explicit stability contract, including formal versioning, thorough documentation, a defined deprecation policy, and often support for backward compatibility. Published interfaces may also require maintaining multiple versions simultaneously and adhering to formal deprecation processes when breaking changes are introduced.[21]

Fragmented chains of function calls are often observed in systems where serverless components (functions) interact with other resources in complex patterns, sometimes described as spaghetti architecture or a distributed monolith. In contrast, systems exhibiting clearer boundaries typically organize serverless components into cohesive groups, where internal public interfaces manage inter-component communication, and published interfaces define communication across group boundaries. This distinction highlights differences in stability guarantees and maintenance commitments, contributing to reduced dependency complexity.[20][21]

Additionally, patterns associated with excessive serverless function chaining are sometimes addressed through architectural strategies that emphasize native service integrations instead of individual functions, a concept referred to as the functionless mindset. However, this approach is noted to involve a steeper learning curve, and integration limitations may vary even within the same cloud vendor ecosystem.[2]

Reporting on serverless databases presents challenges, as retrieving data for a reporting service can either break the bounded contexts, reduce the timeliness of the data, or do both. This applies regardless of whether data is pulled directly from databases, retrieved via HTTP, or collected in batches. Mark Richards refers to this as the *reach-in reporting* anti-pattern.[19] A possible alternative to this approach is for databases to asynchronously push the necessary data to the reporting service instead of the reporting service pulling it. While this method requires a separate contract between services and the reporting service and can be complex to implement, it helps preserve bounded contexts while maintaining a high level of data timeliness.[19]

Principles

Adopting DevSecOps practices can help improve the use and security of serverless technologies.[22]

In serverless applications, the distinction between infrastructure and business logic is often blurred, with applications typically distributed across multiple services. To maximize the effectiveness of testing, integration testing is emphasized for serverless applications.[5] Additionally, to facilitate debugging and implementation, orchestration is used within the bounded context, while choreography is employed between different bounded contexts.[5]

Ephemeral resources are typically kept together to maintain high cohesion. However, shared resources with long spin-up times, such as AWS RDS clusters and landing zones, are often managed in separate repositories, deployment pipeline, and stacks.[5]

See also

- As a service
- Backend as a service
- Cloud computing
- Function as a service

References

1. "ISO/IEC 22123-2:2023 (E) - Information technology — Cloud computing — Part 2: Concepts". *International Standard*: 25.
2. Brisals, Sheen (2024). *Serverless Development on AWS: Building Enterprise-Scale Serverless Solutions*. O'Reilly Media. ISBN 978-1098141936.
3. Emison, Joseph (2023). *Serverless as a Game Changer How to Get the Most Out of the Cloud*. Addison-Wesley Professional. ISBN 9780137392551.
4. *The Software Architect Elevator: Redefining the Architect's Role in the Digital Enterprise*. O'Reilly Media. 2020. ISBN 978-1492077541.
5. Cui, Yan (2020). *Serverless Architectures on AWS* (2nd ed.). Manning. ISBN 978-1617295423.
6. Richards, Mark (March 3, 2020). *Fundamentals of Software Architecture: An Engineering Approach* (1st ed.). O'Reilly Media. ISBN 978-1492043454.
7. Richards, Mark (2021). *Software Architecture: The Hard Parts: Modern Trade-Off Analyses for Distributed Architectures* (1st ed.). O'Reilly Media. ISBN 978-1492086895.

8. *Distributed Tracing in Practice: Instrumenting, Analyzing, and Debugging Microservice*. O'Reilly Media. ISBN 978-1492056638.
9. *Cloud-Native Observability with OpenTelemetry: Learn to gain visibility into systems by combining tracing, metrics, and logging with OpenTelemetry*. ISBN 978-1801077705.
10. Kelly, Daniel; Glavin, Frank G.; Barrett, Enda (2021-08-01). "Denial of wallet—Defining a looming threat to serverless computing" (https://linkinghub.elsevier.com/retrieve/pii/S221421262100079X). *Journal of Information Security and Applications*. **60** 102843. arXiv:2104.08031 (https://arxiv.org/abs/2104.08031). doi:10.1016/j.jisa.2021.102843 (https://doi.org/10.1016%2Fj.jisa.2021.102843). ISSN 2214-2126 (https://search.worldcat.org/issn/2214-2126).
11. "OWASP Serverless Top 10 | OWASP Foundation" (https://owasp.org/www-project-serverless-top-10/). *owasp.org*. Retrieved 2024-05-20.
12. *OWASP/Serverless-Top-10-Project* (https://github.com/OWASP/Serverless-Top-10-Project), OWASP, 2024-05-02, retrieved 2024-05-20
13. Aske, Austin; Zhao, Xinghui (2018-08-13). "Supporting Multi-Provider Serverless Computing on the Edge" (https://doi.org/10.1145/3229710.3229742). *Proceedings of the 47th International Conference on Parallel Processing Companion*. ICPP Workshops '18. New York, NY, USA: Association for Computing Machinery. pp. 1–6. doi:10.1145/3229710.3229742 (https://doi.org/10.1145%2F3229710.3229742). ISBN 978-1-4503-6523-9. S2CID 195348799 (https://api.semanticscholar.org/CorpusID:195348799).
14. Baarzi, Ataollah Fatahi; Kesidis, George; Joe-Wong, Carlee; Shahrad, Mohammad (2021-11-01). "On Merits and Viability of Multi-Cloud Serverless" (https://doi.org/10.1145/3472883.3487002). *Proceedings of the ACM Symposium on Cloud Computing*. SoCC '21. New York, NY, USA: Association for Computing Machinery. pp. 600–608. doi:10.1145/3472883.3487002 (https://doi.org/10.1145%2F3472883.3487002). ISBN 978-1-4503-8638-8. S2CID 239890130 (https://api.semanticscholar.org/CorpusID:239890130).
15. Zhao, Haidong; Benomar, Zakaria; Pfandzelter, Tobias; Georgantas, Nikolaos (2022-12-06). "Supporting Multi-Cloud in Serverless Computing" (https://ieeexplore.ieee.org/document/10061782/references#references). *2022 IEEE/ACM 15th International Conference on Utility and Cloud Computing (UCC)*. pp. 285–290. arXiv:2209.09367 (https://arxiv.org/abs/2209.09367). doi:10.1109/UCC56403.2022.00051 (https://doi.org/10.1109%2FUCC56403.2022.00051). ISBN 978-1-6654-6087-3. S2CID 252383217 (https://api.semanticscholar.org/CorpusID:252383217).
16. *Serverless Computing: Principles and Paradigms*. Springer. 12 May 2023. ISBN 978-3031266324.
17. Foster, Ian; Gannon, Dennis B. (29 September 2017). *Cloud Computing for Science and Engineering (Scientific and Engineering Computation)*. MIT Press. ISBN 978-0262037242.
18. Hellerstein, Joseph; Faleiro, Jose; Gonzalez, Joseph; Schleier-Smith, Johann; Screekanti, Vikram; Tumanov, Alexey; Wu, Chenggang (2019), *Serverless Computing: One Step Forward, Two Steps Back*, arXiv:1812.03651 (https://arxiv.org/abs/1812.03651)
19. Richards, Mark (2015). *Microservices AntiPatterns and Pitfalls*. O'REILLY.
20. "TECHNOLOGY RADAR VOL. 21 An opinionated guide to technology" (https://www.thoughtworks.com/content/dam/thoughtworks/documents/radar/2019/11/tr_technology_radar_vol_21_en.pdf) (PDF). *Technology Radar*. **21**. ThoughtWorks.
21. Fowler, Martin (March–April 2002). "Public versus Published Interfaces" (https://martinfowler.com/ieeeSoftware/published.pdf) (PDF). *IEEE Software*. **19** (2): 18–19. doi:10.1109/52.991326 (https://doi.org/10.1109%2F52.991326).
22. Katzer, Jason (2020). *Learning Serverless: Design, Develop, and Deploy with Confidence*. O'Reilly Media. ISBN 978-1492057017.

Further reading

- Roberts, Mike (25 July 2016). "Serverless Architectures" (http://martinfowler.com/articles/serverless.html). *MartinFowler.com*. Retrieved 30 July 2016.
- Jamieson, Frazer (4 September 2017). "Losing the server? Everybody is talking about serverless architecture" (http://www.bcs.org/content/conWebDoc/58491). *BCS, the Chartered Institute for IT*. Retrieved 7 November 2017.
- Anderson, David (9 March 2022). "Power the Future and Accelerate Your Organization to the Modern Cloud and Serverless with 'The Value Flywheel Effect' " (https://theserverlessedge.com/the-value-flywheel-effect/). *The Serverless Edge*. Retrieved 9 March 2022.

- 14 authors from UC Berkeley (9 February 2019). "Cloud Programming Simplified: A Berkeley View on Serverless Computing[1]".

1. Jonas, Eric (February 2019). "Cloud Programming Simplified: A Berkeley View on Serverless Computing". pp. 1–33. arXiv:1902.03383 (https://arxiv.org/abs/1902.03383) [cs.OS (https://arxiv.org/archive/cs.OS)].

Retrieved from "https://en.wikipedia.org/w/index.php?title=Serverless_computing&oldid=1375122930"